Project 1: Chapters 1-4 (Secure Software Design)

Avijit Saha

Secure Software Development (MSCS-535-B01)

Dr. Yousef Nijim

March 15, 2026

GitHub Link: https://github.com/ovi-saha/MSCS-535-B01_Project_1

1. Project Objective

The project aims to create a secure web application that protects against social engineering attacks such as phishing, as well as technical attacks such as SQL injection. It will ensure that even if the login credentials are stolen, the attacker will still need a second factor, i.e., TOTP, to access the sensitive information over the HTTPS channel.

2. Security Implementation Details

- **Network Security(HTTP):** The application is configured to run on port 8443 using a self-signed SSL certificate.

```
spring.application.name=secureapp
server.port=8443
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=password
server.ssl.key-store-type=PKCS12

spring.datasource.url=jdbc:h2:mem:secure_db
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console


spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

Figure 1: Properties file Setup

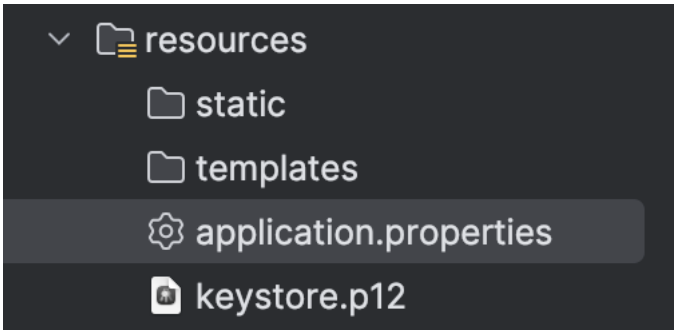

Figure 2: keystore.p12 file is a resource

- **Data Integrity (SQL Injection Defense):** The application used Spring Data JPA and prepared statements for all database operations.

```java
package com.example.secureapp.repository;
import com.example.secureapp.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

Figure 3: Uses of JPA repository in UserRepository

3. Step-by-Step Testing & Verification

User Registration: The first user registers by entering their email and password. In the Postman tool, we can see the registration confirmed in **Figure 4.**

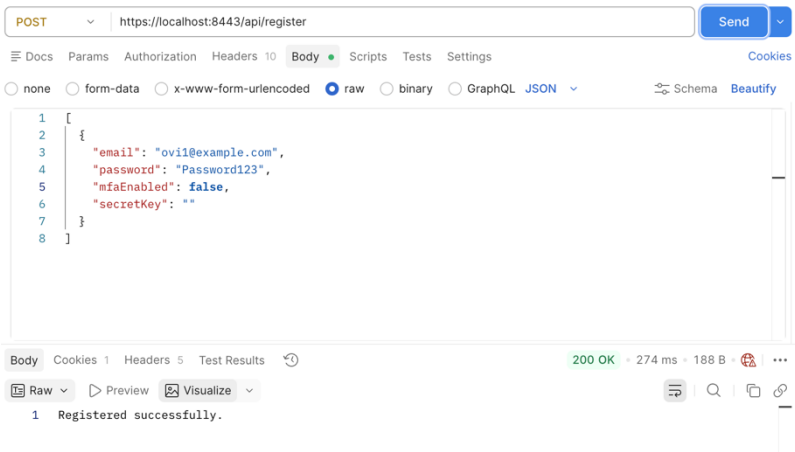

Figure 4: User Registered Successfully

Pre-MFA (Multifactor Authentication) Access Denial: After registration, the user must enable multifactor authentication; otherwise, access is denied, as shown in **Figure 5**.

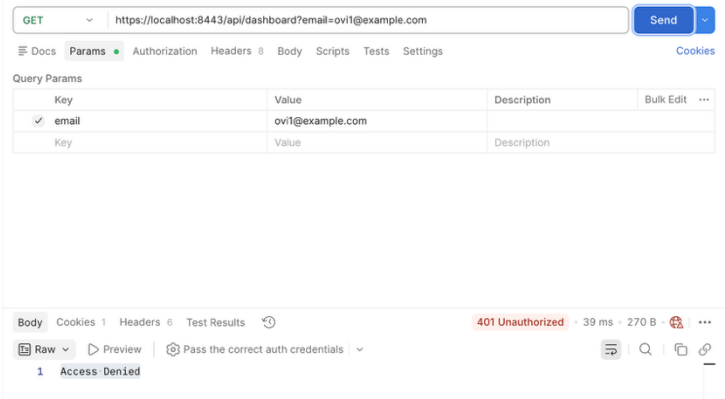

Figure 5: MFA required

User Login: The user can log in with their email and password, but this only partially authenticates them. After the MFA, they can log in completely.

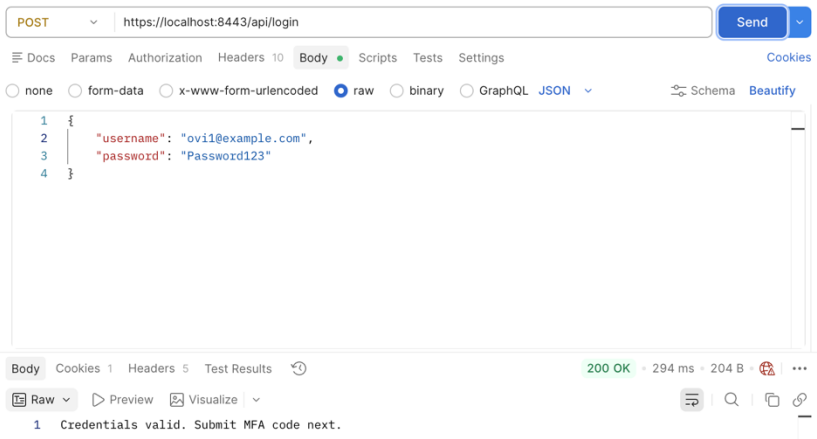

Figure 6: User Login needs MFA to complete the login.

QR code Generation and Verification: The user will generate a QR code for MFA, shown in **Figure 7**. After scanning the QR code in the Google Authenticator app, it will generate a six-digit code to verify the user's identity. When the user enters the six-digit code, they will be verified, as shown in **Figure 8**.

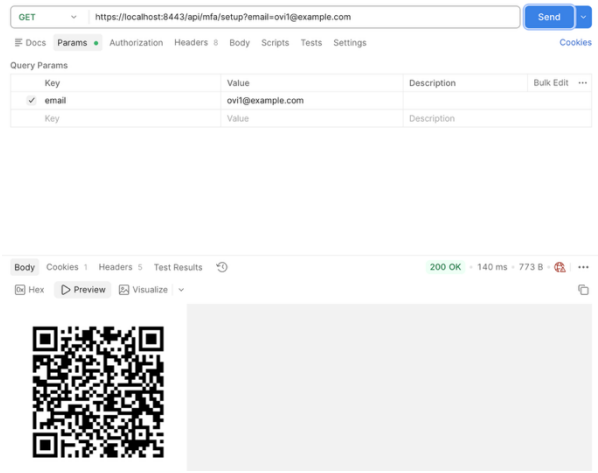

Figure 7: QR code generation

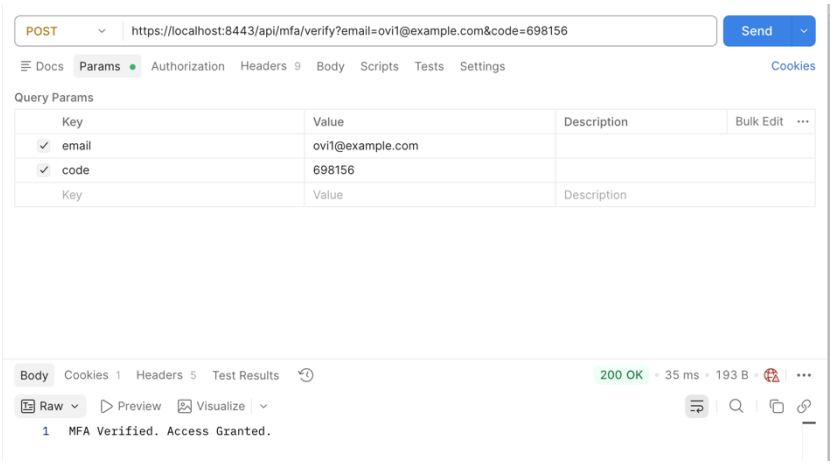

Figure 8: MFA Verified after entering the code

Secure Access Granted: After the MFA, the user can securely log in to the system, and their login is granted, as shown in **Figure 9**.

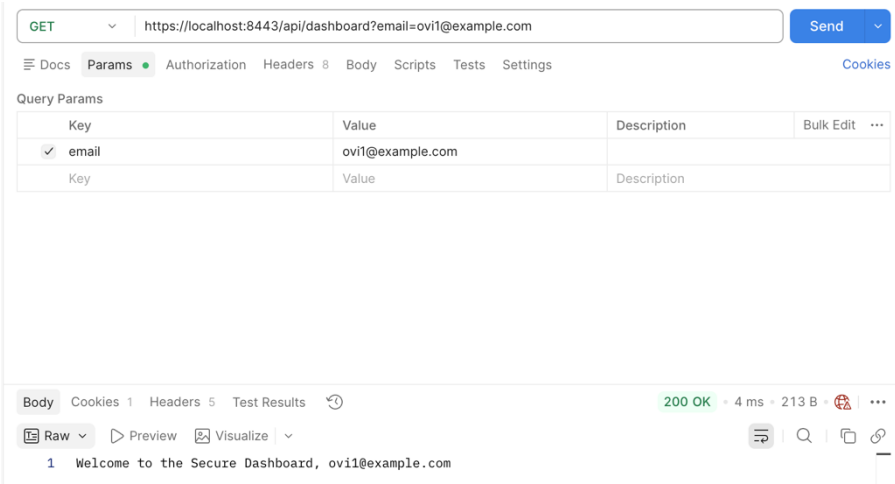

Figure 9: User verified and securely logged in

H2-Console Database: We can connect to the H2-Console database at https://localhost:8443/h2-console to view the database and verify whether MFA is enabled, as shown in **Figure 10.**

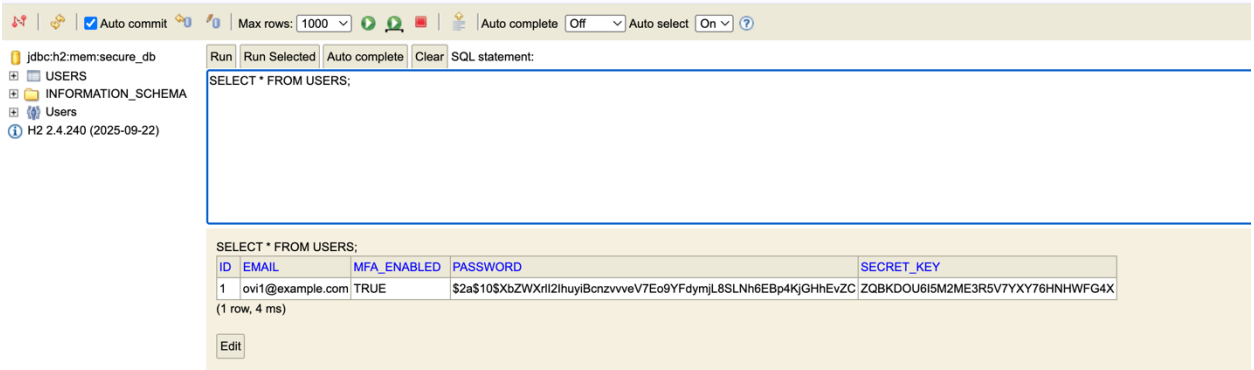

Figure 10: Database showing MFA enabled 'TRUE'